FLOYD WARSHALL ALGORITHM:

1. Definition

The **Floyd–Warshall algorithm** is a dynamic programming algorithm used to find the **shortest paths between all pairs of vertices** in a weighted graph. It can handle negative edge weights, but it does not work correctly when the graph contains a negative-weight cycle.

2. Algorithm:

1. Start with the distance matrix of the graph.
2. Set the distance from each vertex to itself as 0.
3. Set the distance between directly connected vertices to their edge weights.
4. If there is no direct edge between two vertices, set their distance to infinity (INF).
5. Consider each vertex as an intermediate vertex.
6. Update the distance using the formula:

$$dist[i][j] = \min(d, ist[i][j]\ dist[i][k] + dist[k][j])$$

7. Repeat for all intermediate vertices k.
8. Display the final distance matrix containing the shortest distances between all pairs of vertices.

3. Time Complexity:

Time Complexity: $O(V^3)$

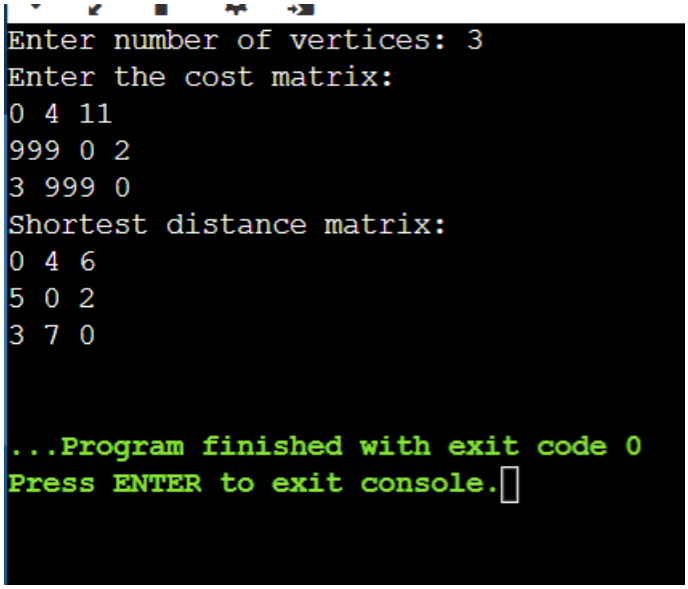
The algorithm uses three nested loops to check every pair of vertices through each possible intermediate vertex.

Here, $V$is the number of vertices.

4. Space Complexity:

Space Complexity: $O(V^2)$

5. OUTPUT:



6. Conclusion

The Floyd–Warshall algorithm is a dynamic programming algorithm used to find the shortest paths between all pairs of vertices in a weighted graph. It updates the distance matrix by checking whether a path through an intermediate vertex is shorter than the existing path. It is useful in network routing, road maps, and transportation systems.